University of Toronto Scarborough
CSCC24 2023 Summer Final Examination
August 2023
Duration - 3 hours
You need at least 35% on this exam to pass the course.
Aids allowed: 2 aid sheets (letter size or A4, double-sided, don't hand in).

1:	/ 5	5:	/12
2:	/ 8	6:	/10
3:	/23	7:	/13
4:	/10	8:	/10
		Total:	/91

There is an appendix of techinical reminders at the back.
There are blank pages for scratch work after the questions.

1. [5 marks] Miscellaenous short questions.

   (a) [2 marks] The following is functional pseudocode in a Haskell-like language but not necessarily with Haskell scoping rules.

```
let title = "vector" in
let pt = \x -> title ++ " " ++ x in
let title = "null" in pt "space"
```

   What is the result of evaluation if:

   - the scoping rule is lexical scoping? "vector space"
   - the scoping rule is dynamic scoping? "null space"

   (b) [3 marks] For a functional language that does dynamic typing but no recursion, complete rewriting the following mutual recursion and use case with the diagonal trick.

```
let f = \n -> if n=0 then 2 else 3 * g (n-1)
    g = \n -> if n=0 then 1 else f (n-1) + 1
in f 10
```

```
let mkf = \[mkf mkg] n -> if n=0 then 2 else 3 * mkg [mkf mkg] (n-1)
    mkg = \[mkf mkg] n -> if n=0 then 1 else 1 + mkf [mkf mkg] (n-1)
in mkf [mkf mkg]          10
```

2. [8 marks] In this question, we represent a polynomial by `[Double]`, for example $3 - 1x + 4x^2$ is represented as `[3, -1, 4]`.
So the list item at index i (0-based indexing) is the coefficient for x^i. (This can work the same way for infinite series using infinite lists.)

   For simplicity, we allow redundant representations, e.g., `[3, -1, 4, 0, 0]` also represents $3 - 1x + 4x^2$. The zero polynomial also has multiple representations: `[]`, `[0]`, `[0, 0]`, etc.

   (a) [2 marks] Implement polynomial addition under this representation.

```
add :: [Double] -> [Double] -> [Double]
```

$p(x) = a + bx + cx^2$, $q(x) = d + ex$ $\Big\}$ $p + q = (a+d) + (b+e)x + cx^2$

```
add as [] = as
add [] bs = bs
add (a:at) (b:bt) = (a+b) : add at bt
```

   (b) [1 mark] Implement multiplying a polynomial by x. (This will be helpful for the next part.)

```
oneUp :: [Double] -> [Double]
```

```
oneUp xs = 0 : xs        oneUp = (0:)
```

   (c) [5 marks] Implement multiplying two polynomials. Suggestions:

```
map :: (a -> b) -> [a] -> [b]
```

$(a + xr(x))p(x) = ap(x) + xr(x)p(x)$ (r = $r(x)$)

```
mul :: [Double] -> [Double] -> [Double]
```

```
mul [] _ = []
```

oneUp (mul r p)

r(x) · (x · p(x))

```
mul (a:r) p = (map (*a) p) `add` (mul r (oneUp p))
mul (x:xs) ys = add (map (*x) ys) (mul xs (oneUp ys))
```

3. [23 marks] Induction, lazy evaluation, and successive approximations. The following definitions are given (you may have seen g from a past test):

```
f a xs = a : h a xs  where  h a (x:xt) = f (a+x) xt
                            h a [] = []

g a (x:xt) = a : g (a+x) xt
g a [] = a : []

-- For part (c)
s i = i : s (i+1)
hd (x:xs) = x
hd [] = error "empty list"
```

(a) [2 marks] What are the values of the following two expressions? No steps required.

f a [b, c] = a : (a+b) : (a+b+c) : []
(f (a+b) [c]; f (a+b+c) [])

g a [b, c] = a : (a+b) : (a+b+c) : []

(b) [6 marks] Prove by structural induction: For all finite list xs, for all a: $f\ a\ xs = g\ a\ xs$

Basis: xs = [], a arbitrary

f a [] = a : h a [] by defn of f
= a : [] " " " h
= g a [] " " " g.

Induction step: let xs = x : xt st

for all a, f a xt = g a xt (IH).

wts f a xs = g a xs

Then f a xs = a : h a (x:xt) by defn of f
= a : f (a+x) xt " " " h
= a : g (a+x) xt by IH
= g a (x:xt)
= g a xs. ∎

$s\ i = i : s\ (i+1)$

(c) [7 marks] But f and g differ under lazy evaluation. Show the lazy evaluation steps for the following two expressions until you get 0:

```
hd (f 0 (s 1))
```

-> hd (0 : h 0 (s 1))

→ 0.

```
hd (g 0 (s 1))
```

-> hd (g 0 (1 : s (1+1)))

→ hd (0 : g (0+1) (s (1+1)))

→ 0.

(d) [8 marks] f and g can also differ when making infinite lists:

```
fps = f 1 fps
```

```
gps = g 1 gps
```

Calculate their successive approximations up to and including fps_2 and gps_2.

$fps_0 = \bot$

$fps_1 = f\ 1\ fps_0$
$= f\ 1\ \bot$
$= 1 : h\ 1\ \bot$
$= 1 : \bot.$

$fps_2 = f\ 1\ fps_1$
$= f\ 1\ (1 : \bot)$
$= 1 : h\ \underset{a}{1}\ (\underset{x}{1} : \underset{xt}{\bot})$
$= 1 : f\ (1+1)\ \bot$
$= 1 : (1+1) : h\ (1+1)\ \bot$
$= 1 : (1+1) : \bot.$

$gps_0 = \bot$

$gps_1 = g\ 1\ gps_0$
$= g\ 1\ \bot$
$= \bot.$

$gps_2 = g\ 1\ gps_1$
$= g\ 1\ \bot$
$= \bot.$

4. [10 marks] Folds.

(a) [5 marks] `split` from the Midterm Test could be implemented directly this way:

```
split "" = []
split (c:cs) = if c == ':'
               then "" : split cs
               else case split cs of y:yt -> (c:y) : yt
                                     [] -> [[c]]
```

"abc:def" → "abc":"def"

Express it in terms of `foldr`.

```
split = foldr op z
  where
    z = []

    op c r = if c == ':'
             then "" : r
             else case r of y:yt -> (c:y) : yt
                            [] -> [[c]]
```

acc = split cs

```
acc = []
for c in cs:
    if c == ':':
        acc = "" : acc
    else:
        acc = (case ct of y:yt -> ...
                          ...)
acc
```

(b) [5 marks] The type `S` below is intended to be a wrapper for sorted lists. (You will carefully write code to make that true.)

```
data S a = MkS [a]
unS :: S a -> [a]  -- Unwrapper.
unS (MkS xs) = xs
```

We can use it to trick `foldMap` into doing sorting! Make `S a` an instance of Semigroup and Monoid, and complete `sort` such that it sorts.

```
sort :: Ord a => [a] -> [a]
sort xs = unS (foldMap (\x -> MkS [x]           ) xs)


instance Ord a => Monoid (S a) where
    -- mempty :: S a
    mempty = MkS []


instance Ord a => Semigroup (S a) where
    -- (<>) :: S a -> S a -> S a
    MkS xs <> MkS ys = MkS (helper xs ys)

-- What should helper do?
-- You may assume the inputs are sorted;
-- you must ensure the output is sorted.
helper :: Ord a => [a] -> [a] -> [a]
```

```
helper [] ys = ys
helper xs [] = xs

helper (x:xt) (y:yt) = if x < y
                       then x : helper xt (y:yt)
                       else y : helper (x:xt) yt.
```

```
[3,1,2] --map--> [MkS [3], MkS [1], MkS [2]]

MkS a <> MkS b <> MkS c
= sorted ([1,2,3])
```


"inefficient mergesort" aka insertion sort

5. [12 marks] Type inference.

   (a) [5 marks] Show the type inference steps for the following. We assume that the type of 9 is Integer. Note that we begin with having the variable `m` in the environment. (You should get a type mismatch error.)

```
infer {m :: Maybe Bool} (case m of Nothing -> m
                                   Just b  -> Just 9) :
```

(b) [7 marks] Show the type inference steps for the following.

`infer {} (\f -> Just (f True)) :`

create unknown uf

Tb := infer {f :: uf} (Just (f True))

Te1 := infer {f :: uf} (f True)

Tf := infer {f :: uf} f

return uf

Te2 := infer {f :: uf} True

return Bool

create unknown u0

unify Tf (Te2 → u0)

unify-intern uf (Bool → u0)

return u0

Table:

uf := Bool → u0

Te1 = u0

return (apply subst Maybe Te1) = Maybe u0

Tb = Maybe u0

return (apply-subst uf → Tb)

= (Bool → u0) → Maybe u0.

6. [10 marks] Parametric polymorphism.

(a) [2 marks] Victor wrote a function with the following type and test result:

```
e0 :: (Int -> a) -> [a]
e0 id = [3, 1, 4]            -- id = \x -> x
```

Predict the answer of `e0 (\x -> x + 1)`.

[4, 2, 5]

(b) [8 marks] In fact, that works in general. Prove that $e :: \forall a \cdot (Int \to a) \to [a]$ satisfies

$$map\ f_R\ (e\ id) = e\ f_R \qquad (\text{for all } A_R,\ f_R :: Int \to A_R)$$

$\Updownarrow$ $e\ id\ \langle [a] \rangle\ e\ f_R$ with $\langle a \rangle = f_R$

length same and each $\langle a \rangle$ — can think "congruence modulo f_R"

Parametricity says:

$e\ \langle \forall a.\ (Int \to a) \to [a] \rangle\ e$

Expand:

for all A_L, A_R, relation $\sim$ between them

let $\langle a \rangle = \sim$ in

$e_{A_L}\ \langle (Int \to a) \to [a] \rangle\ e_{A_R}$

Expand:

for all A_L, A_R, relation $\sim$ between them

let $\langle a \rangle = \sim$ in

for all $f_L :: Int \to A_L$, $f_R :: Int \to A_R$

if $f_L\ \langle Int \to a \rangle\ f_R$

then $e_{A_L}\ f_L\ \langle [a] \rangle\ e_{A_R}\ f_R$

Expand:

if $f_L\ x\ \langle a \rangle\ f_R\ x$

then $e_{A_L}\ f_L\ \langle [a] \rangle\ e_{A_R}\ f_R$

let $f_L = id$, $A_L = Int$,

$\sim = \langle a \rangle$ = the relation s.t. $x\ \langle a \rangle\ y$ iff $y = f_R\ x = f_R$

wts $id\ x\ \langle a \rangle\ f_R\ x$ for all x

$\Leftrightarrow x\ \langle a \rangle\ f_R\ x$.

We have $id\ x\ \langle a \rangle\ f_R\ x$ since $id\ x = x \Rightarrow f_R\ (id\ x) = f_R\ x$.

∴ by parametricity, $e\ id\ \langle [a] \rangle\ e\ f_R$.

7. [13 marks] Grammar and parsing.

   (a) [3 marks] We begin with the following ambiguous grammar (start symbol F):

```
F ::= F OP F
    | "~" F
    | "(" F ")"
    | Var
OP ::= "&&" | "=>"
Bit ::= "0" | "1"
```

   Draw two different parse trees for `"0 && 1 => 0"`.

(b) [10 marks] The ambiguity is to be resolved by the following precedence table from highest to lowest.

`( )`	
unary `~`	
`&&`	left associative
`=>`	right associative

Implement a parser accordiningly. The data type to parse to is:

```
data F = Bit Bool
       | Not F        -- unary ~
       | And F F      -- &&
       | Imp F F      -- =>

-- Fill in this so the grader knows where to start.
mainParser :: Parser F
mainParser = whitespace *>                    <* eof
```

8. [10 marks] We will model a toy language that has two mutable Int variables X and Y. Hence the representation is a state transition function from (old X, old Y) to (new X, new Y, answer):

```
data XY a = MkXY ((Int,Int) -> (Int,Int,a))
unXY :: XY a -> ((Int,Int) -> (Int,Int,a))    -- unwrapper
unXY (MkXY f) = f

-- We assume that these connectives are available.
pure :: a -> XY a
(>>=) :: XY a -> (a -> XY b) -> XY b
(*>) :: XY a -> XY b -> XY b
```

(a) [2 marks] Implement these primitives for reading and writing X. (Y is similar and omitted.)

```
getX :: XY Int
getX = MkXY (\(x,y) ->                          )

setX :: Int -> XY ()
setX i = MkXY (\(x,y) ->                        )
```

(b) [8 marks] The toy language is represented by:

```
data Stmt = SetX Expr           -- store value of expr in X
          | SetY Expr           -- store value of expr in Y
          | Seq Stmt Stmt       -- sequential composition, 2 stmts
          | WhileXNotZero Stmt  -- while X!=0 do ...

data Expr = Lit Int
          | VarX             -- read X
          | VarY             -- read Y
          | Add Expr Expr    -- binary plus
```

Implement the following cases of the interpreter. (Others cases are similar, omitted.)

```
evalExpr :: Expr -> XY Int

evalExpr (Lit i) =

evalExpr VarX =

interp :: Stmt -> XY ()

interp (SetX expr) =


interp (WhileXNotZero s) =
```

(End of questions.)

Blank page for sketch work.

Blank page for sketch work.

Blank page for sketch work.

Blank page for sketch work.

Appendix

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr op z [] = z
foldr op z (x:xs) = op x (foldr op z xs)
```

b : result type / acc type
a : input type

```
class Semigroup a where
  (<>) :: a -> a -> a

class Semigroup a => Monoid a where
  mempty :: a

class Foldable f where
  foldMap :: Monoid m => (a -> m) -> f a -> m
  -- and others
```

Parsing:

```
-- Connectives.
fmap :: (a -> b) -> Parser a -> Parser b
pure :: a -> Parser a
liftA2 :: (a -> b -> c) -> Parser a -> Parser b -> Parser c
(<*>) :: Parser (a -> b) -> Parser a -> Parser b
(*>) :: Parser a -> Parser b -> Parser b
(<*) :: Parser a -> Parser b -> Parser a
(>>=) :: Parser a -> (a -> Parser b) -> Parser b
empty :: Parser a
(<|>) :: Parser a -> Parser a -> Parser a

-- Read a natural number (non-negative integer), then skip trailing spaces.
natural :: Parser Integer

-- Read the wanted operator, then skip trailing spaces.
operator :: String -> Parser String

-- Parentheses
openParen, closeParen :: Parser Char

-- For right-associative operators.
chainr1 :: Parser a -> Parser (a -> a -> a) -> Parser a

-- For left-associative operators.
chainl1 :: Parser a -> Parser (a -> a -> a) -> Parser a
```

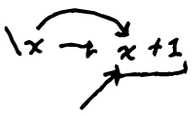
Some type inference rules:

```
infer env (case eM of Nothing -> e0
                      Just v  -> e1) :
    T := infer env eM
    create unknown u
    unify T (Maybe u)

    T0 := infer env e0

    T1 := infer (env plus v::u) e1

    unify T0 T1
    return (apply-subst T0)

infer env (\v -> expr):
    create unknown uv (for v)
    Tb := infer (env plus v::uv) expr
    return (apply-subst (uv -> Tb))

infer env (f e):
    Tf := infer env f
    Te := infer env e
    create new unknown u
    unify Tf (Te -> u)
    return (apply-subst u)
```

f :: ∀a. [a] → Int

infer {f :: ∀a. [a] → Int} f

Tf := ∀a. [a] → Int
instantiate Tf with new unknown u0
return [u0] → Int



create unknown ux
Tb := infer {x :: ux} (x+1)
return (apply-subst ux → Tb)
= Int → Int

Table:
ux := Int

let id = (\x → x)

u0 → u0 → generalize to ∀a. a → a

in [(id 1, id 'a')] {id :: ∀a. a → a}

---

x + 1] term (expr)

(Int → u0) → u1] type (expr)

u0 := Bool
u1 := Int → Bool

type substitution
apply-subst (Int → u0) → u1
= (Int → Bool) → (Int → Bool)

env = {x :: Int, f :: u0 → Int}

infer {x :: Int, f :: u0 → Int} x
return Int

infer {x :: Int, f :: u0 → Int} y
undefined var error

Table:
u0 := Bool

unify T0 T1
unify-intern (apply-subst T0) (apply-subst T1)
= Bool → Int

Updated table:

e :: ∀a. [a] → [a]

e <∀a. [a] → [a]> e